Serialization Formats

Simon Frost

Overview

RDFLib.jl supports reading and writing RDF data in many formats. This vignette demonstrates each format with the same sample graph.

```
using RDFLib

# Build a sample graph
g = RDFGraph()
ex = Namespace("http://example.org/")
bind!(g, "ex", ex)

add!(g, Triple(ex("earth"), RDF.type, ex("Planet")))
add!(g, Triple(ex("earth"), RDFS("label"), Literal("Earth"; lang="en")))
add!(g, Triple(ex("earth"), RDFS("label"), Literal("Terre"; lang="fr")))
add!(g, Triple(ex("earth"), ex("radius_km"), Literal(6371)))
add!(g, Triple(ex("earth"), ex("hasMoon"), ex("moon")))
add!(g, Triple(ex("moon"), RDF.type, ex("Moon")))
add!(g, Triple(ex("moon"), RDFS("label"), Literal("The Moon")))
```

```
RDFGraph (7 triples)
```

Turtle

Turtle is the most human-readable RDF format. It supports prefixes, abbreviations, and multi-value shorthand.

```
ttl = serialize(g, TurtleFormat())
println(ttl)
```

```
@prefix ex: <http://example.org/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:earth a ex:Planet ;
    ex:hasMoon ex:moon ;
    ex:radius_km 6371 ;
    rdfs:label "Earth"@en,
        "Terre"@fr .

ex:moon a ex:Moon ;
    rdfs:label "The Moon" .
```

```
# Round-trip: parse back
g2 = parse_rdf(ttl, TurtleFormat())
println("Round-trip: $(length(g2)) triples (original: $(length(g)))")
```

```
Round-trip: 7 triples (original: 7)
```

N-Triples

N-Triples is the simplest line-based format — one triple per line, fully expanded URIs.

```
nt = serialize(g, NTriplesFormat())
println(nt)
```

```
<http://example.org/earth> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://example
<http://example.org/earth> <http://www.w3.org/2000/01/rdf-schema#label> "Earth"@en .
<http://example.org/earth> <http://www.w3.org/2000/01/rdf-schema#label> "Terre"@fr .
<http://example.org/earth> <http://example.org/radius_km> "6371"^^<http://www.w3.org/2001/XM
<http://example.org/earth> <http://example.org/hasMoon> <http://example.org/moon> .
<http://example.org/moon> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://example.
<http://example.org/moon> <http://www.w3.org/2000/01/rdf-schema#label> "The Moon" .
```

```
g3 = parse_rdf(nt, NTriplesFormat())
println("Parsed: $(length(g3)) triples")
```

```
Parsed: 7 triples
```

N3 (Notation3)

N3 extends Turtle with formulas (quoted graphs), variables, and implications — the foundation for N3 reasoning.

```
n3_str = serialize_n3(g)
println(n3_str)
```

```
@prefix ex: <http://example.org/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:earth a ex:Planet ;
    ex:hasMoon ex:moon ;
    ex:radius_km 6371 ;
    rdfs:label "Earth"@en, "Terre"@fr .

ex:moon a ex:Moon ;
    rdfs:label "The Moon" .
```

RDF/XML

The original RDF serialization format, based on XML.

```
rdfxml = serialize(g, RDFXMLFormat())
println(rdfxml)
```

```
<?xml version="1.0" encoding="UTF-8"?>
<rdf:RDF xmlns:ex="http://example.org/" xmlns:owl="http://www.w3.org/2002/07/owl#" xmlns:rdf=
rdf-syntax-ns#" xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#" xmlns:skos="http://www.w3
  <rdf:Description rdf:about="http://example.org/earth">
    <rdf:type rdf:resource="http://example.org/Planet"/>
    <rdfs:label xml:lang="en">Earth</rdfs:label>
    <rdfs:label xml:lang="fr">Terre</rdfs:label>
    <ex:radius_km rdf:datatype="http://www.w3.org/2001/XMLSchema#integer">6371</ex:radius_km
    <ex:hasMoon rdf:resource="http://example.org/moon"/>
  </rdf:Description>
  <rdf:Description rdf:about="http://example.org/moon">
```

```
    <rdf:type rdf:resource="http://example.org/Moon"/>
    <rdfs:label>The Moon</rdfs:label>
  </rdf:Description>
</rdf:RDF>
```

```
g4 = parse_rdf(rdfxml, RDFXMLFormat())
println("Parsed: $(length(g4)) triples")
```

```
Parsed: 7 triples
```

JSON-LD

JSON-LD embeds RDF in JSON, making it web-developer friendly.

```
jsonld = serialize(g, JSONLDFormat())
println(jsonld)
```

```
{
    "@context": {
        "rdfs": "http://www.w3.org/2000/01/rdf-schema#",
        "owl": "http://www.w3.org/2002/07/owl#",
        "skos": "http://www.w3.org/2004/02/skos/core#",
        "ex": "http://example.org/",
        "rdf": "http://www.w3.org/1999/02/22-rdf-syntax-ns#",
        "xsd": "http://www.w3.org/2001/XMLSchema#"
    },
    "@graph": [
        {
            "rdfs:label": [
                {
                    "@language": "en",
                    "@value": "Earth"
                },
                {
                    "@language": "fr",
                    "@value": "Terre"
                }
            ],
            "@id": "http://example.org/earth",
            "ex:hasMoon": {
```

```
                "@id": "http://example.org/moon"
            },
            "ex:radius_km": 6371,
            "@type": "http://example.org/Planet"
        },
        {
            "rdfs:label": "The Moon",
            "@id": "http://example.org/moon",
            "@type": "http://example.org/Moon"
        }
    ]
}
```

```
# Parse JSON-LD
g5 = parse_rdf(jsonld, JSONLDFormat())
println("Parsed: $(length(g5)) triples")
```

```
Parsed: 7 triples
```

JSON-LD Processing

```
# Expand - remove context, show full URIs
expanded = jsonld_expand(jsonld)
println("Expanded:")
println(expanded)
```

```
Expanded:
Dict{String, Any}[Dict("@graph" => Any[Dict{String, Any}("rdfs:label" => Any[Dict{String, Any
```

```
# Compact - apply a context to shorten URIs
context = Dict{String,Any}("@context" => Dict{String,Any}("ex" => "http://example.org/", "lab
compacted = jsonld_compact(jsonld, context)
println("Compacted:")
println(compacted)
```

```
Compacted:
Dict{String, Any}("@context" => Dict{String, Any}("@context" => Dict{String, Any}("label" =>
schema#label", "ex" => "http://example.org/")), "@graph" => Any[Dict{String, Any}("rdfs:label
```

N-Quads

N-Quads extend N-Triples with a fourth element for named graphs.

```
ds = Dataset()
for t in g
    add!(ds, t)
end
nq = serialize_nquads(ds)
println(nq)
```

```
<http://example.org/earth> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://example
<http://example.org/earth> <http://www.w3.org/2000/01/rdf-schema#label> "Earth"@en .
<http://example.org/earth> <http://www.w3.org/2000/01/rdf-schema#label> "Terre"@fr .
<http://example.org/earth> <http://example.org/radius_km> "6371"^^<http://www.w3.org/2001/XM
<http://example.org/earth> <http://example.org/hasMoon> <http://example.org/moon> .
<http://example.org/moon> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://example.
<http://example.org/moon> <http://www.w3.org/2000/01/rdf-schema#label> "The Moon" .
```

TriG

TriG extends Turtle with named graph support.

```
trig = serialize_trig(ds)
println(trig)
```

```
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

{
    ns1:earth a ns1:Planet ;
        ns1:hasMoon ns1:moon ;
        ns1:radius_km 6371 ;
        rdfs:label "Earth"@en, "Terre"@fr .

    ns1:moon a ns1:Moon ;
        rdfs:label "The Moon" .
}
```

TriX

TriX is an XML-based format for named graphs.

```
trix = serialize_trix(g)
println(trix)
```

```
<?xml version="1.0" encoding="utf-8"?>
<TriX xmlns="http://www.w3.org/2004/03/trix/trix-1/">
  <graph>
    <triple>
      <uri>http://example.org/earth</uri>
      <uri>http://www.w3.org/1999/02/22-rdf-syntax-ns#type</uri>
      <uri>http://example.org/Planet</uri>
    </triple>
    <triple>
      <uri>http://example.org/earth</uri>
      <uri>http://www.w3.org/2000/01/rdf-schema#label</uri>
      <plainLiteral xml:lang="en">Earth</plainLiteral>
    </triple>
    <triple>
      <uri>http://example.org/earth</uri>
      <uri>http://www.w3.org/2000/01/rdf-schema#label</uri>
      <plainLiteral xml:lang="fr">Terre</plainLiteral>
    </triple>
    <triple>
      <uri>http://example.org/earth</uri>
      <uri>http://example.org/radius_km</uri>
      <typedLiteral datatype="http://www.w3.org/2001/XMLSchema#integer">6371</typedLiteral>
    </triple>
    <triple>
      <uri>http://example.org/earth</uri>
      <uri>http://example.org/hasMoon</uri>
      <uri>http://example.org/moon</uri>
    </triple>
    <triple>
      <uri>http://example.org/moon</uri>
      <uri>http://www.w3.org/1999/02/22-rdf-syntax-ns#type</uri>
      <uri>http://example.org/Moon</uri>
    </triple>
    <triple>
      <uri>http://example.org/moon</uri>
```

```
      <uri>http://www.w3.org/2000/01/rdf-schema#label</uri>
      <plainLiteral>The Moon</plainLiteral>
    </triple>
  </graph>
</TriX>
```

HexTuples (HEXT)

A line-based JSON format, one array per triple.

```
hext = serialize_hextuples(g)
println(hext)
```

```
["http://example.org/earth","http://www.w3.org/1999/02/22-rdf-syntax-ns#type","http://example
["http://example.org/earth","http://www.w3.org/2000/01/rdf-schema#label","Earth","http://www
rdf-syntax-ns#langString","en",""]
["http://example.org/earth","http://www.w3.org/2000/01/rdf-schema#label","Terre","http://www
rdf-syntax-ns#langString","fr",""]
["http://example.org/earth","http://example.org/radius_km","6371","http://www.w3.org/2001/XM
["http://example.org/earth","http://example.org/hasMoon","http://example.org/moon","globalId
["http://example.org/moon","http://www.w3.org/1999/02/22-rdf-syntax-ns#type","http://example
["http://example.org/moon","http://www.w3.org/2000/01/rdf-schema#label","The Moon","http://w
```

Long Turtle

Verbose Turtle with one triple per line (useful for diffs).

```
lt = serialize(g, TurtleFormat())
println(lt)
```

```
@prefix ex: <http://example.org/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

ex:earth a ex:Planet ;
    ex:hasMoon ex:moon ;
    ex:radius_km 6371 ;
```

```
    rdfs:label "Earth"@en,
        "Terre"@fr .

ex:moon a ex:Moon ;
    rdfs:label "The Moon" .
```

RDF Patch

RDF Patch represents changes (additions/deletions) to a graph.

```
# serialize_rdfpatch takes additions and deletions vectors
additions = collect(g)
deletions = Triple[]
patch = serialize_rdfpatch(additions, deletions)
println(first(patch, 500))
```

```
TX .
A <http://example.org/earth> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://exampl
A <http://example.org/earth> <http://www.w3.org/2000/01/rdf-schema#label> "Earth"@en .
A <http://example.org/earth> <http://www.w3.org/2000/01/rdf-schema#label> "Terre"@fr .
A <http://example.org/earth> <http://example.org/radius_km> "6371"^^<http://www.w3.org/2001/
A <http://example.org/earth> <http://example.org/hasMoon> <http://example.org/moon> .
A <http://exa
```

File I/O

Saving to Files

```
# Format is auto-detected from file extension
save_rdf(g, "planets.ttl")          # Turtle
save_rdf(g, "planets.nt")           # N-Triples
save_rdf(g, "planets.jsonld")       # JSON-LD
save_rdf(g, "planets.rdf")          # RDF/XML

# Or specify format explicitly
save_rdf(g, "planets.dat"; format=NTriplesFormat())
```

Loading from Files

```
# Format auto-detected from extension
g = load_rdf_file("planets.ttl")

# Explicit format
g = load_rdf_file("planets.dat"; format=NTriplesFormat())
```

Content Negotiation

```
# Get MIME types for formats
println("Turtle:    ", mime_type(TurtleFormat()))
println("JSON-LD:   ", mime_type(JSONLDFormat()))
println("RDF/XML:   ", mime_type(RDFXMLFormat()))
println("N-Triples: ", mime_type(NTriplesFormat()))
```

```
Turtle:    text/turtle
JSON-LD:   application/ld+json
RDF/XML:   application/rdf+xml
N-Triples: application/n-triples
```

Format Comparison

Format	Human Readable	Compact	Named Graphs	Standard
Turtle				W3C
N-Triples				W3C
N3			(formulas)	W3C
RDF/XML				W3C
JSON-LD				W3C
TriG				W3C
N-Quads				W3C
TriX				W3C
HexTuples				Community